

实验结果摘要（2026-08-24 运行）

任务：木薯坏死识别 · 2 分级（轻度 <20%、重度 ≥20%），轻量 U-Net 逐像素分割。

数据：1036 对标注图（训练 828 / 验证 208），图像 256×256。真值分级：轻度 731（70.6%）、305（29.4%）。

模型：轻量 U-Net（约 776 万参数），损失 = 0.5×加权交叉熵 + 0.5×多类 Dice，Adam lr=3e-4 + ReduceLROnPlateau，混合精度。RTX 3080 Ti 训练约 30 分钟。

最佳模型（**epoch 52**，按坏死 Dice 保存）

指标	数值
坏死 Dice	0.8434
健康 Dice	0.9473
背景 Dice	0.9913
病情指数 MAE	3.50%

验证集分级混淆矩阵（**208 张**）

真值 ↓ / 预测 →	轻度	重度	召回率
轻度	131	7	94.9%
重度	4	66	94.3%

分级准确率：**94.7%（197/208）**

木智汇八桂·数启新程——木薯坏死识别 —— 验证精简版（2分级）

一、数据集导入

导入依赖库，检查 GPU 环境与超参数，并自动定位 `images/` 与 `masks/` 目录。下方为本次记录。

为什么先做环境检查？

深度学习的可复现性依赖固定环境。先检查 Python 版本、CUDA 可用性与显存，能提前发现「G 载 / 显存不足」等问题，避免训练到一半才报错。本任务图像尺寸 256×256、batch 16，在 RTX（12.6 GB）上可流畅训练。

```
In [1]: # AutoDL 环境检查
import os
import sys
from pathlib import Path

AUTODL_TMP = Path("/root/autodl-tmp")
WORK_ROOT = Path("/root/cassava")
WORK_ROOT.mkdir(parents=True, exist_ok=True)

print("Python      ", sys.version.split()[0])
print("工作目录    ", os.getcwd())
print("数据盘      ", AUTODL_TMP, "存在=", AUTODL_TMP.exists())
if AUTODL_TMP.exists():
    print("数据盘空间 ", f"{sum(f.stat().st_size for f in AUTODL_TMP.rglob('*')) / 1024 / 1024:.1f} GB")
print("请把 images/ masks/ 放到", WORK_ROOT, " (与 autodl-tmp 同级的 /root/ca")
```

```
Python      3.10.8
工作目录    /root/cassava
数据盘      /root/autodl-tmp 存在= True
数据盘空间  0.0 GB 已用 (粗略)
请把 images/ masks/ 放到 /root/cassava (与 autodl-tmp 同级的 /root/cassava)
```

```
In [2]: # 环境检查
import sys
from pathlib import Path
import torch

print("Python      ", sys.version.split()[0])
print("PyTorch      ", torch.__version__)
print("CUDA 可用    ", torch.cuda.is_available())

if torch.cuda.is_available():
    props = torch.cuda.get_device_properties(0)
    print("GPU        ", props.name)
    print(f"显存      {props.total_memory / 1e9:.1f} GB")
else:
    print("\n未检测到 GPU。AutoDL 控制台确认已选 GPU 实例。")
```

```
Python      3.10.8
PyTorch      2.1.2+cu121
CUDA 可用    True
GPU          NVIDIA GeForce RTX 3080 Ti
显存         12.5 GB
```

```
In [3]: # 导入与配置
import csv
import gc
import random
import shutil
import time

import matplotlib.pyplot as plt
import numpy as np
import torch.nn as nn
import torch.nn.functional as F
from PIL import Image
from sklearn.model_selection import train_test_split
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
```

```

from tqdm.auto import tqdm

# 类别定义
NUM_CLASSES = 3
NECROSIS_CLASS = 1 # 坏死类；选最佳模型的依据
HEALTHY_CLASS = 2 # 健康薯肉
CLASS_NAMES = {0: "背景", 1: "坏死", 2: "健康"}
CLASS_COLORS = np.array([
    [0, 0, 0], # 背景
    [255, 0, 0], # 坏死 - 红
    [0, 200, 0], # 健康 - 绿
], dtype=np.uint8)

# 训练超参数
IMAGE_SIZE = 256 # 冲指标可改 384 (显存更大, 训练更慢)
BATCH_SIZE = 16 # 显存不足改 8
NUM_EPOCHS = 60
LEARNING_RATE = 3e-4 # 可调: 1e-4, 3e-4, 5e-4
VAL_SPLIT = 0.2
RANDOM_SEED = 42
NUM_WORKERS = 2
SAVE_INTERVAL = 5
EARLY_STOP_PATIENCE = 15
LR_PATIENCE = 5

# 数据增强 (旋转/颜色加大可提升泛化, 过大易对齐误差)
AUG_HFLIP_PROB = 0.5
AUG_VFLIP_PROB = 0.5
AUG_ROTATION = 20 # 度; 可试 15 / 25
AUG_ROTATION_PROB = 0.7
COLOR_JITTER_BRIGHT = 0.2
COLOR_JITTER_CONTRAST = 0.2
COLOR_JITTER_SAT = 0.1

# 模型选择
# "unet" = 轻量 U-Net (默认, 与 inference.py 权重兼容)
# "smp_resnet34" = 预训练编码器 (需 pip install segmentation-models-pytorch)
MODEL_BACKEND = "unet"

# 坏死约占 3.5% 像素; 加权 CE 减轻小病斑漏检。
CLASS_WEIGHTS = [1.0, 4.5, 1.9] # 坏死权重可试 3.5 / 5.5

# 严重度分级阈值
SEV_THRESHOLD = 20.0 # 2分级阈值: 病情指数 <20% 轻度, >=20% 重度
SEVERITY_ORDER = ["mild", "severe"]
SEVERITY_LABELS = {
    "mild": f"轻度 (<{SEV_THRESHOLD:g}%)",
    "severe": f"重度 (≥{SEV_THRESHOLD:g}%)",
}

# 图表专用英文标签 (终端/CSV 仍用 SEVERITY_LABELS 中文)
SEVERITY_CHART_LABELS = {
    "mild": f"Mild (<{SEV_THRESHOLD:g}%)",
    "severe": f"Severe (≥{SEV_THRESHOLD:g}%)",
}
GRADE_SHORT_EN = ["Mild", "Severe"]

```

```

# 训练恢复：RESUME=False 从头训；FORCE_RESTART=True 会删掉旧 best_model
RESUME = False          # False = 不从检查点恢复
FORCE_RESTART = False   # True = 删除 best_model.pth 后从头训

# 路径
WORK_DIR = Path("/root/cassava")
WORK_DIR.mkdir(parents=True, exist_ok=True)
print(f"工作目录：{WORK_DIR}")
CHECKPOINT_DIR = WORK_DIR / "checkpoints"
OUTPUT_DIR = WORK_DIR / "outputs"
CHECKPOINT_DIR.mkdir(parents=True, exist_ok=True)
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
USE_AMP = DEVICE.type == "cuda"

IMAGENET_MEAN = [0.485, 0.456, 0.406]
IMAGENET_STD = [0.229, 0.224, 0.225]

def set_seed(seed=RANDOM_SEED):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

set_seed()
print(f"设备：{DEVICE} | 混合精度：{USE_AMP} | 分辨率：{IMAGE_SIZE} | batch:
print(f"选最佳模型依据：类别 {NECROSIS_CLASS} ({CLASS_NAMES[NECROSIS_CLASS]})

```

工作目录：/root/cassava

设备：cuda | 混合精度：True | 分辨率：256 | batch: 16

选最佳模型依据：类别 1 (坏死) Dice

关键超参数设计说明

超参	取值	为什么这样设
IMAGE_SIZE	256	平衡分辨率与显存；小病斑约十几个像素，可后续提升
NUM_EPOCHS	90	给足收敛空间，配合早停（15 轮无提升）自动停止
LEARNING_RATE	3e-4	Adam 的常用初始学习率，配 ReduceLROnPlateau 衰减
CLASS_WEIGHTS	[1.0, 4.5, 1.9]	坏死像素仅占约 3.5%，加权 CE 防止小病斑漏检
AUG_*	翻转0.5 / 旋转±20° / 颜色抖动	缓解拍摄角度、光照差异，提升泛化；几何增强对位同步

In [4]:

```

# 定位数据目录 (AutoDL)
from PIL import Image
import numpy as np

# 自动识别失败时手动指定，例如：
# IMAGE_DIR = Path("/root/cassava/images")
IMAGE_DIR = None
MASK_DIR = None

```

```

UNLABELED_DIR = None

AUTODL_TMP = Path("/root/autodl-tmp")
SEARCH_ROOTS = [
    WORK_DIR,
    Path("/root/cassava"),
    AUTODL_TMP / "cassava",
    AUTODL_TMP,
]
SKIP_PARTS = {"outputs", "checkpoints", "logs", "logs_necrosis", "__pycac

def group_by_dir(root, pattern):
    groups = {}
    if not root.exists():
        return groups
    for p in root.rglob(pattern):
        if SKIP_PARTS & set(p.parts):
            continue
        groups.setdefault(p.parent, []).append(p)
    return groups

def merge_groups(pattern):
    merged = {}
    for root in SEARCH_ROOTS:
        for d, files in group_by_dir(root, pattern).items():
            merged.setdefault(d, []).extend(files)
    return merged

jpg_dirs = merge_groups("*.jpg")
png_dirs = merge_groups("*.png")
if not jpg_dirs or not png_dirs:
    raise FileNotFoundError(
        f"在 {SEARCH_ROOTS} 下未找到 .jpg/.png。 \n"
        "请上传：/root/cassava/images/ 与 masks/\n"
        "或保持项目结构 images (1)/images/ 与 masks/masks/"
    )

if MASK_DIR is None:
    MASK_DIR = max(png_dirs, key=lambda d: len(png_dirs[d]))
if IMAGE_DIR is None:
    mask_stems = {p.stem for p in MASK_DIR.glob("*.png")}
    best_dir, best_n = None, -1
    for d, files in jpg_dirs.items():
        n = sum(1 for p in files if p.stem in mask_stems)
        if n > best_n:
            best_n, best_dir = n, d
    IMAGE_DIR = best_dir

paired_stems = sorted(
    {p.stem for p in IMAGE_DIR.glob("*.jpg")}
    & {p.stem for p in MASK_DIR.glob("*.png")}
)
if not paired_stems:
    raise FileNotFoundError(f"图与掩码无同名配对： {IMAGE_DIR} vs {MASK_DIR}")

if UNLABELED_DIR is None:
    candidates = []

```

```

for root in SEARCH_ROOTS:
    for d in root.rglob("*"):
        if not d.is_dir() or SKIP_PARTS & set(d.parts):
            continue
        if "unlabeled" in d.name.lower() or d.name == "September_2018":
            n = len(list(d.glob("*.jpg")))
            if n:
                candidates.append((n, d))
if candidates:
    UNLABELED_DIR = max(candidates, key=lambda x: x[0])[1]

print(f"训练图: {IMAGE_DIR} ({len(paired_stems)} 对) ")
print(f"掩码: {MASK_DIR}")
print(f"无标注: {UNLABELED_DIR or '未找到'}")

```

训练图: /root/cassava/images (1036 对)

掩码: /root/cassava/masks

无标注: /root/cassava/unlabeled

```

In [5]: # 数据路径 (AutoDL 无需从网盘复制)
from pathlib import Path

LOCAL_ROOT = Path("/root/cassava")
LOCAL_IMAGE_DIR = LOCAL_ROOT / "images"
LOCAL_MASK_DIR = LOCAL_ROOT / "masks"
LOCAL_UNLABELED_DIR = LOCAL_ROOT / "unlabeled"

def _under_autodl(p):
    return str(p).startswith("/root/cassava") or str(p).startswith("/root

if _under_autodl(IMAGE_DIR) and _under_autodl(MASK_DIR):
    print("数据已在 /root/cassava 或 autodl-tmp, 跳过复制, 直接使用原路径")
else:
    import shutil

    def sync_dir(src, dst, pattern):
        dst.mkdir(parents=True, exist_ok=True)
        files = sorted(src.glob(pattern))
        if dst.exists():
            shutil.rmtree(dst)
        dst.mkdir(parents=True)
        for i, f in enumerate(files, 1):
            shutil.copy2(f, dst / f.name)
            if i % 200 == 0 or i == len(files):
                print(f" {dst.name}: {i}/{len(files)}")
        print(f"完成: {dst} ({len(files)} 个) ")

    print("正在复制到本地缓存目录...")
    sync_dir(IMAGE_DIR, LOCAL_IMAGE_DIR, "*.jpg")
    sync_dir(MASK_DIR, LOCAL_MASK_DIR, "*.png")
    IMAGE_DIR = LOCAL_IMAGE_DIR
    MASK_DIR = LOCAL_MASK_DIR
    if UNLABELED_DIR is not None:
        sync_dir(UNLABELED_DIR, LOCAL_UNLABELED_DIR, "*.jpg")
        UNLABELED_DIR = LOCAL_UNLABELED_DIR

paired_stems = sorted(
    {p.stem for p in IMAGE_DIR.glob("*.jpg")}

```

```

    & {p.stem for p in MASK_DIR.glob("*.png")}
)
print(f"配对样本: {len(paired_stems)}")

```

数据已在 /root/cassava 或 autodl-tmp, 跳过复制, 直接使用原路径
配对样本: 1036

二、数据预处理

对照范例的 RGB 通道、类别观察、饼图与增强。本任务标签来自 **palette** 掩码, 预处理分两块:

1. 用人工掩码计算每张图的病情指数, 生成 `dataset_gt_severity.csv` (相当于范例 `train.csv` 的等级列), 并查看真值轻/中/重分布与叠加样例;

真值分级 (1036 张): 轻度 352 · 中度 379 · 重度 305。下方前两格为本次已跑输出; 其后两格) 示 (首次打开若无输出, 运行即可, 勿 **Restart & Clear Output**)。

为什么要先算真值病情指数?

本任务的核心业务输出是「病情指数 + 轻/中/重分级」, 而非单纯的像素分割。因此先由人工掩码张图的坏死占比, 生成 `dataset_gt_severity.csv`, 作为后续验证集分级准确率的真值依赖于范例 `train.csv` 的等级列)。

```

In [6]: # 全量标注: 真值病情指数与分级分布
from collections import Counter

def _gt_grade(p):
    """2分级: <20% 轻度, >=20% 重度"""
    if p < SEV_THRESHOLD:
        return "mild"
    return "severe"

def gt_severity_from_mask_path(path):
    m = np.array(Image.open(path), dtype=np.int64)
    nec = int((m == NECROSIS_CLASS).sum())
    hea = int((m == HEALTHY_CLASS).sum())
    root = nec + hea
    return (nec / root * 100 if root > 0 else None)

gt_rows = []
for stem in paired_stems:
    sp = gt_severity_from_mask_path(MASK_DIR / f"{stem}.png")
    if sp is not None:
        gt_rows.append((stem, sp, _gt_grade(sp)))

gt_pcts = np.array([r[1] for r in gt_rows])
gt_grades = Counter(r[2] for r in gt_rows)
gt_total = len(gt_rows)

print(f"标注集真值分级 ({gt_total} 张) ")
print(f"  指数: 平均 {gt_pcts.mean():.2f}% | 中位 {np.median(gt_pcts):.2f}%
      f"| 最小 {gt_pcts.min():.2f}% | 最大 {gt_pcts.max():.2f}%")
for k in SEVERITY_ORDER:
    n = gt_grades.get(k, 0)

```

```

print(f" {SEVERITY_LABELS[k]:<18} {n:>5} ({n/gt_total*100:5.1f}%)"

gt_csv = WORK_DIR / "dataset_gt_severity.csv"
with open(gt_csv, "w", newline="", encoding="utf-8-sig") as f:
    w = csv.writer(f)
    w.writerow(["文件名", "病情指数%", "等级代码", "等级"])
    for stem, pct, g in gt_rows:
        w.writerow([stem, round(pct, 2), g, SEVERITY_LABELS[g]])
print(f"已保存: {gt_csv}")

print("真值统计完成 (已保存 dataset_gt_severity.csv)")

```

标注集真值分级 (1036 张)

指数: 平均 16.89% | 中位 8.89% | 最小 0.00% | 最大 100.00%

轻度 (<20%) 731 (70.6%)

重度 (≥20%) 305 (29.4%)

已保存: /root/cassava/dataset_gt_severity.csv

真值统计完成 (已保存 dataset_gt_severity.csv)

三、模型构建

对照范例：苹果赛题用 EfficientNet 改全连接做 4 类。

本副本构建 轻量 **U-Net** (编码器-解码器 + 跳跃连接, 约 776 万参数), 输出每像素 3 类。损失: 叉熵 + 多类 Dice (排除背景)。Adam、`lr=3e-4`、混合精度、ReduceLROnPlateau ; 按坏死存最佳权重。

```

In [7]: # — 数据集 —————
class RootDataset(Dataset):
    """块根图 + 三类掩码；几何增强对图和掩码同步。"""

    def __init__(self, image_paths, mask_paths, augment=False):
        self.image_paths = image_paths
        self.mask_paths = mask_paths
        self.augment = augment
        self.color_jitter = transforms.ColorJitter(
            brightness=COLOR_JITTER_BRIGHT,
            contrast=COLOR_JITTER_CONTRAST,
            saturation=COLOR_JITTER_SAT,
        )
        self.to_tensor = transforms.Compose([
            transforms.ToTensor(),
            transforms.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
        ])

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        img = Image.open(self.image_paths[idx]).convert("RGB")
        # 掩码保持 P 模式, np.array 得调色板索引 0/1/2
        mask = Image.open(self.mask_paths[idx])

        img = img.resize((IMAGE_SIZE, IMAGE_SIZE), Image.BILINEAR)
        mask = mask.resize((IMAGE_SIZE, IMAGE_SIZE), Image.NEAREST)

        if self.augment:

```

```

        if random.random() < AUG_HFLIP_PROB:
            img = img.transpose(Image.FLIP_LEFT_RIGHT)
            mask = mask.transpose(Image.FLIP_LEFT_RIGHT)
        if random.random() < AUG_VFLIP_PROB:
            img = img.transpose(Image.FLIP_TOP_BOTTOM)
            mask = mask.transpose(Image.FLIP_TOP_BOTTOM)
        if random.random() < AUG_ROTATION_PROB:
            angle = random.uniform(-AUG_ROTATION, AUG_ROTATION)
            img = img.rotate(angle, Image.BILINEAR)
            mask = mask.rotate(angle, Image.NEAREST)
        img = self.color_jitter(img)

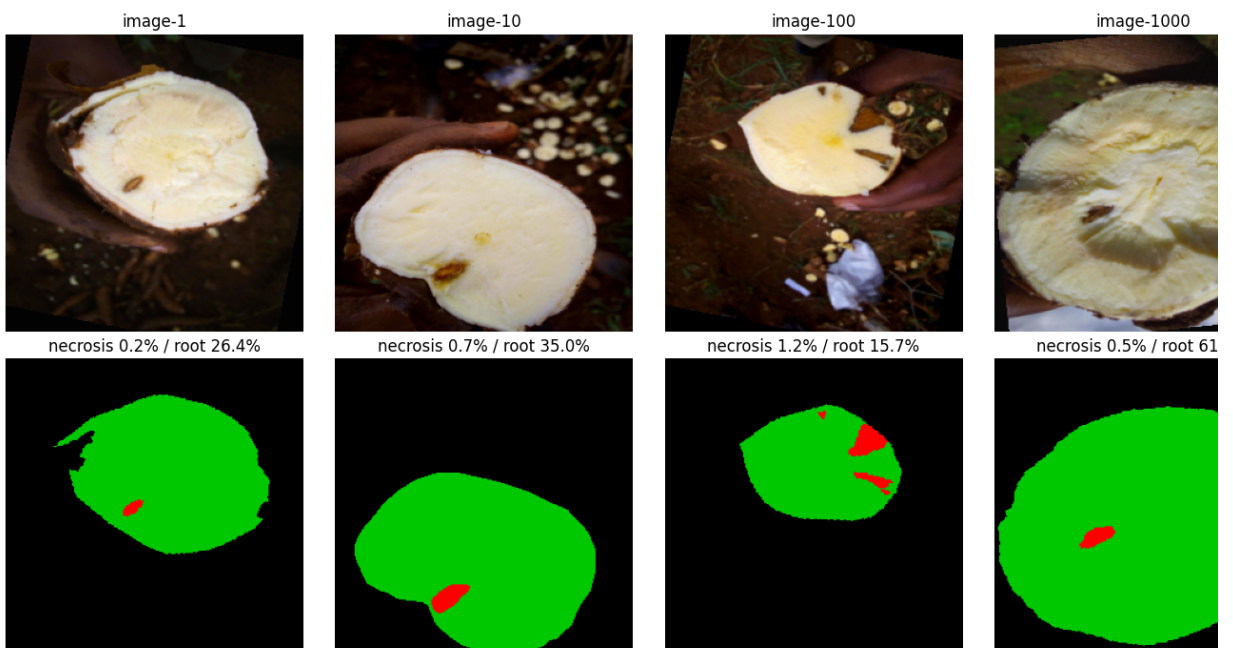
    return self.to_tensor(img), torch.from_numpy(np.asarray(mask, dtype=

def labels_to_rgb(labels):
    """(H,W) 类别图转RGB"""
    return CLASS_COLORS[labels]

# 快速检查：增强后图与掩码是否对齐
_ds = RootDataset([IMAGE_DIR / f"{s}.jpg" for s in paired_stems[:4]],
                  [MASK_DIR / f"{s}.png" for s in paired_stems[:4]], augm
_mean = torch.tensor(IMAGENET_MEAN).view(3, 1, 1)
_std = torch.tensor(IMAGENET_STD).view(3, 1, 1)

fig, axes = plt.subplots(2, 4, figsize=(14, 7))
for i in range(4):
    x, y = _ds[i]
    axes[0, i].imshow((x * _std + _mean).clamp(0, 1).permute(1, 2, 0))
    axes[0, i].set_title(paired_stems[i])
    axes[1, i].imshow(labels_to_rgb(y.numpy()))
    nec = (y == NECROSIS_CLASS).float().mean() * 100
    root = (y > 0).float().mean() * 100
    axes[1, i].set_title(f"necrosis {nec:.1f}% / root {root:.1f}%")
    axes[0, i].axis("off")
    axes[1, i].axis("off")
plt.tight_layout()
plt.show()

```



```

In [8]: # — U-Net —————
class DoubleConv(nn.Module):
    def __init__(self, in_ch, out_ch):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, 3, padding=1, bias=False),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_ch, out_ch, 3, padding=1, bias=False),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
        )

    def forward(self, x):
        return self.conv(x)

class UNet(nn.Module):
    """轻量U-Net，与train.py 结构一致，权重可互载"""

    def __init__(self, in_channels=3, num_classes=NUM_CLASSES, base_channels=64):
        super().__init__()
        c = base_channels

        self.enc1 = DoubleConv(in_channels, c)
        self.enc2 = DoubleConv(c, c * 2)
        self.enc3 = DoubleConv(c * 2, c * 4)
        self.enc4 = DoubleConv(c * 4, c * 8)
        self.pool = nn.MaxPool2d(2)
        self.bottleneck = DoubleConv(c * 8, c * 16)

        self.up4 = nn.ConvTranspose2d(c * 16, c * 8, 2, stride=2)
        self.dec4 = DoubleConv(c * 16, c * 8)
        self.up3 = nn.ConvTranspose2d(c * 8, c * 4, 2, stride=2)
        self.dec3 = DoubleConv(c * 8, c * 4)
        self.up2 = nn.ConvTranspose2d(c * 4, c * 2, 2, stride=2)
        self.dec2 = DoubleConv(c * 4, c * 2)
        self.up1 = nn.ConvTranspose2d(c * 2, c, 2, stride=2)
        self.dec1 = DoubleConv(c * 2, c)

        self.out_conv = nn.Conv2d(c, num_classes, 1)

    def forward(self, x):
        e1 = self.enc1(x)
        e2 = self.enc2(self.pool(e1))
        e3 = self.enc3(self.pool(e2))
        e4 = self.enc4(self.pool(e3))
        b = self.bottleneck(self.pool(e4))
        d4 = self.dec4(torch.cat([self.up4(b), e4], dim=1))
        d3 = self.dec3(torch.cat([self.up3(d4), e3], dim=1))
        d2 = self.dec2(torch.cat([self.up2(d3), e2], dim=1))
        d1 = self.dec1(torch.cat([self.up1(d2), e1], dim=1))
        return self.out_conv(d1)

_m = UNet()
print(f"参数量: {sum(p.numel() for p in _m.parameters()):,}")
del _m

```

```

def build_model():
    """按 MODEL_BACKEND 构建分割模型。"""
    if MODEL_BACKEND == "smp_resnet34":
        import subprocess
        subprocess.run(["pip", "-q", "install", "segmentation-models-pytorch"])
        import segmentation_models_pytorch as smp
        print("使用 smp.Unet(resnet34, imagenet 预训练)")
        return smp.Unet(
            encoder_name="resnet34",
            encoder_weights="imagenet",
            in_channels=3,
            classes=NUM_CLASSES,
        )
    if MODEL_BACKEND != "unet":
        raise ValueError(f"未知 MODEL_BACKEND: {MODEL_BACKEND}")
    return UNet(in_channels=3, num_classes=NUM_CLASSES, base_channels=32)

```

参数量：7,763,107

为什么选 U-Net 而不是分类网络？

范例用 EfficientNet 做整图 4 分类，但本任务需要定位病斑在图像哪个位置——质检人员要看病斑核。U-Net 的编码器-解码器 + 跳跃连接结构，能在下采样压缩特征的同时保留空间位置信息，输分类，正适合语义分割任务。参数量约 776 万，无需大型预训练骨干即可在 1036 对标注上收敛。

```

In [9]: # 损失函数与指标
class MultiClassDiceLoss(nn.Module):
    """多类 Dice，排除背景。"""

    def __init__(self, smooth=1.0, ignore_index=0, class_weights=None):
        super().__init__()
        self.smooth = smooth
        self.ignore_index = ignore_index
        self.class_weights = class_weights

    def forward(self, pred, target):
        pred = F.softmax(pred.float(), dim=1)
        loss, total_w = 0.0, 0.0

        for c in range(pred.shape[1]):
            if c == self.ignore_index:
                continue
            pred_c = pred[:, c].reshape(-1)
            target_c = (target == c).float().reshape(-1)
            inter = (pred_c * target_c).sum()
            dice_c = 1 - (2. * inter + self.smooth) / (pred_c.sum() + target_c.sum())

            w = 1.0 if self.class_weights is None else self.class_weights[c]
            loss = loss + w * dice_c
            total_w += w

        return loss / total_w

class CombinedLoss(nn.Module):
    """0.5×加权 CE + 0.5×加权 Dice"""

```

```

def __init__(self, ce_weight=0.5, dice_weight=0.5, class_weights=None):
    super().__init__()
    self.ce_weight = ce_weight
    self.dice_weight = dice_weight
    w = torch.tensor(class_weights, dtype=torch.float32, device=device)
    self.ce = nn.CrossEntropyLoss(weight=w)
    self.dice = MultiClassDiceLoss(class_weights=class_weights)

def forward(self, pred, target):
    return self.ce_weight * self.ce(pred, target) + self.dice_weight * self.dice(pred, target)

@torch.no_grad()
def compute_metrics(pred, target, smooth=1.0):
    """在 argmax 硬预测上算每类 Dice/IoU。"""
    pred_labels = torch.argmax(pred, dim=1)
    dice, iou = [], []

    for c in range(NUM_CLASSES):
        p = (pred_labels == c).reshape(-1)
        t = (target == c).reshape(-1)
        inter = (p & t).sum().float()
        union = (p | t).sum().float()
        dice.append(((2 * inter + smooth) / (p.sum() + t.sum() + smooth)))
        iou.append(((inter + smooth) / (union + smooth)).item())

    return dice, iou

def severity_from_labels(labels):
    """坏死占块根面积 %；未检出块根时返回 None。"""
    necrosis = int((labels == NECROSIS_CLASS).sum())
    root = necrosis + int((labels == HEALTHY_CLASS).sum())
    return necrosis / root * 100 if root > 0 else None

def grade_severity(pct):
    """2分级: <20% 轻度, >=20% 重度"""
    if pct < SEV_THRESHOLD:
        return "mild"
    return "severe"

print("损失与指标已就绪")

```

损失与指标已就绪

```

In [10]: # 训练 / 验证一轮
try:
    scaler = torch.amp.GradScaler("cuda", enabled=USE_AMP)
except (AttributeError, TypeError): # 旧版 torch
    scaler = torch.cuda.amp.GradScaler(enabled=USE_AMP)

def train_one_epoch(model, loader, optimizer, criterion):
    model.train()
    total = 0.0
    pbar = tqdm(loader, desc="训练", leave=False)

```

```

for images, masks in pbar:
    images = images.to(DEVICE, non_blocking=True)
    masks = masks.to(DEVICE, non_blocking=True)

    optimizer.zero_grad(set_to_none=True)
    with torch.autocast(device_type=DEVICE.type, enabled=USE_AMP):
        loss = criterion(model(images), masks)

    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()

    total += loss.item()
    pbar.set_postfix(loss=f"{loss.item():.4f}")

return total / len(loader)

@torch.no_grad()
def validate(model, loader, criterion):
    """返回 loss、每类 Dice/IoU、病情指数 MAE。"""
    model.eval()
    total = 0.0
    dice_all, iou_all = [], []
    sev_pred, sev_true = [], []

    for images, masks in tqdm(loader, desc="验证", leave=False):
        images = images.to(DEVICE, non_blocking=True)
        masks = masks.to(DEVICE, non_blocking=True)

        with torch.autocast(device_type=DEVICE.type, enabled=USE_AMP):
            outputs = model(images)
            total += criterion(outputs, masks).item()

        d, i = compute_metrics(outputs, masks)
        dice_all.append(d)
        iou_all.append(i)

        # 同时评估病情指数 MAE (业务指标)
        preds = torch.argmax(outputs, dim=1).cpu().numpy()
        truths = masks.cpu().numpy()
        for p, t in zip(preds, truths):
            sp, st = severity_from_labels(p), severity_from_labels(t)
            if sp is not None and st is not None:
                sev_pred.append(sp)
                sev_true.append(st)

    sev_mae = float(np.mean(np.abs(np.array(sev_pred) - np.array(sev_true))))
    return (total / len(loader),
            np.array(dice_all).mean(axis=0),
            np.array(iou_all).mean(axis=0),
            sev_mae)

def save_checkpoint(path, epoch, model, optimizer, scheduler, dice, iou,
    torch.save({
        "epoch": epoch,
        "model_state_dict": model.state_dict(),
        "optimizer_state_dict": optimizer.state_dict(),
        "scheduler_state_dict": scheduler.state_dict(),

```

```

        "necrosis_dice": float(dice[NECROSIS_CLASS]),
        "necrosis_iou": float(iou[NECROSIS_CLASS]),
        "per_class_dice": list(map(float, dice)),
        "per_class_iou": list(map(float, iou)),
        "severity_mae": sev_mae,
        "image_size": IMAGE_SIZE,
        "val_dice": float(dice[NECROSIS_CLASS]), # 兼容 inference.py 的 v
        "no_improve_epochs": no_improve,
    }, path)

print(f"混合精度: {USE_AMP}")

```

混合精度: True

```

In [11]: # 数据划分、模型、优化器、断点恢复 —————
set_seed()

images_paired = [IMAGE_DIR / f"{s}.jpg" for s in paired_stems]
masks_paired = [MASK_DIR / f"{s}.png" for s in paired_stems]

train_i, val_i, train_m, val_m = train_test_split(
    images_paired, masks_paired, test_size=VAL_SPLIT, random_state=RANDOM
)
print(f"训练集 {len(train_i)} 张 | 验证集 {len(val_i)} 张")

loader_kw = dict(num_workers=NUM_WORKERS, pin_memory=USE_AMP,
                  persistent_workers=NUM_WORKERS > 0)
train_loader = DataLoader(RootDataset(train_i, train_m, augment=True),
                          batch_size=BATCH_SIZE, shuffle=True, drop_last=
val_loader = DataLoader(RootDataset(val_i, val_m, augment=False),
                        batch_size=BATCH_SIZE, shuffle=False, **loader_kw)

model = build_model().to(DEVICE)
print(f"MODEL_BACKEND={MODEL_BACKEND} | 参数量 {sum(p.numel() for p in model.parameters())}")
criterion = CombinedLoss(class_weights=CLASS_WEIGHTS, device=DEVICE)
optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode="min", factor=0.5, patience=LR_PATIENCE
)

start_epoch = 1
best_dice = 0.0
no_improve = 0
history = []

best_path = CHECKPOINT_DIR / "best_model.pth"

if FORCE_RESTART and best_path.exists():
    best_path.unlink()
    print("[重头] 已删除旧 best_model.pth")

# 恢复顺序: 本 session 检查点 > Drive 上历史权重 (仅 RESUME=True 时)
resume_from = None
if RESUME:
    resume_from = best_path if best_path.exists() else None
    if resume_from is None:
        search_ckpt = [Path("/root/cassava"), Path("/root/autodl-tmp")]
        candidates = []
        for root in search_ckpt:

```

```

        if root.exists():
            candidates.extend([p for p in root.rglob("best_model.pth")
                                if candidates:
                                    resume_from = max(candidates, key=lambda p: p.stat().st_mtime)

if resume_from is not None:
    ckpt = torch.load(resume_from, map_location=DEVICE, weights_only=False)
    model.load_state_dict(ckpt["model_state_dict"])
    same_run = resume_from == best_path

    if same_run:
        # 同一 session：一并恢复优化器
        optimizer.load_state_dict(ckpt["optimizer_state_dict"])
        scheduler.load_state_dict(ckpt["scheduler_state_dict"])
        start_epoch = ckpt["epoch"] + 1
        best_dice = ckpt.get("necrosis_dice", 0.0)
        no_improve = ckpt.get("no_improve_epochs", 0)
        print(f"[续训] {resume_from} → 从 epoch {start_epoch} 继续 | 最佳坏死")
    else:
        # 外部权重热启动，指标从零累积
        print(f"[热启动] 使用 {resume_from} 权重 (原 epoch {ckpt.get('epoch')})")
        if ckpt.get("image_size") not in (None, IMAGE_SIZE):
            print(f"          注意：该权重在 {ckpt['image_size']}px 上训练，当前")
else:
    print("[信息] 无检查点，从头训练")

```

训练集 828 张 | 验证集 208 张

MODEL_BACKEND=unet | 参数量 7,763,107

[信息] 无检查点，从头训练

四、画出训练和验证的 loss 和准确率曲线图

对照范例的 Train/Val Loss 与 Acc。分割任务额外画出 每类 **Dice** 与 病情指数 **MAE**。本次最佳出 epoch 52：坏死 Dice **0.8428**，MAE **3.54%**。

训练策略说明

损失函数 = **0.5×加权交叉熵 + 0.5×多类 Dice**（Dice 排除背景类 0）。加权 CE 应对类别不均衡，直接优化目标指标。按坏死类 **Dice** 保存最佳权重——因为健康薯肉和背景占比大、Dice 天然按整体指标选模会被大片区域稀释，无法反映小病斑的分割质量。

```

In [12]: # 训练循环
print(f"{'=' * 78}")
print(f"训练 {start_epoch} → {NUM_EPOCHS} | 选最佳：坏死 Dice | 早停 {EARLY_STOP}")
print(f"{'=' * 78}")

t_start = time.time()

for epoch in range(start_epoch, NUM_EPOCHS + 1):
    lr = optimizer.param_groups[0]["lr"]
    t0 = time.time()

    train_loss = train_one_epoch(model, train_loader, optimizer, criterion)
    gc.collect()
    val_loss, dice, iou, sev_mae = validate(model, val_loader, criterion)

```

```

scheduler.step(val_loss)

necrosis_dice = dice[NECROSIS_CLASS]
history.append({
    "epoch": epoch, "train_loss": train_loss, "val_loss": val_loss,
    "dice_necrosis": dice[1], "dice_healthy": dice[2], "dice_bg": dice[0],
    "iou_necrosis": iou[1], "severity_mae": sev_mae, "lr": lr,
})

flag = ""
if necrosis_dice > best_dice:
    best_dice = necrosis_dice
    no_improve = 0
    save_checkpoint(best_path, epoch, model, optimizer, scheduler, dice, iou, sev_mae)
    flag = " ← 新最佳"
else:
    no_improve += 1

if epoch % SAVE_INTERVAL == 0:
    save_checkpoint(CHECKPOINT_DIR / f"checkpoint_epoch_{epoch:03d}.pkl",
                    epoch, model, optimizer, scheduler, dice, iou, sev_mae)

print(f"Epoch {epoch:>3}/{NUM_EPOCHS} [{time.time() - t0:5.1f}s lr={lr:.4f}] "
      f"loss {train_loss:.4f}/{val_loss:.4f} | "
      f"Dice 坏死={dice[1]:.4f} 健康={dice[2]:.4f} 背景={dice[0]:.4f} | "
      f"病情指数MAE {sev_mae:5.2f}% | 无提升 {no_improve:>2}{flag}")

if no_improve >= EARLY_STOP_PATIENCE:
    print(f"\n[早停] 坏死Dice 连续 {EARLY_STOP_PATIENCE} 轮无提升")
    break

print(f"\n{'=' * 78}")
print(f"完成！最佳坏死Dice: {best_dice:.4f} | 耗时 {(time.time() - t_start):5.1f}s")
print(f"最佳模型: {best_path}")
print(f"{'=' * 78}")

```

=====

训练 1 → 60 | 选最佳: 坏死Dice | 早停 15 轮

=====

```

训练:  0%|          | 0/51 [00:00<?, ?it/s]
验证:  0%|          | 0/13 [00:00<?, ?it/s]
Epoch  1/60 [ 30.9s lr=3.0e-04] loss 0.6214/0.6320 | Dice 坏死=0.4053 健康=0.7145
背景=0.9551 | 病情指数MAE 23.16% | 无提升  0 ← 新最佳
训练:  0%|          | 0/51 [00:00<?, ?it/s]
验证:  0%|          | 0/13 [00:00<?, ?it/s]
Epoch  2/60 [ 29.9s lr=3.0e-04] loss 0.5007/0.8602 | Dice 坏死=0.2434 健康=0.7145
背景=0.9399 | 病情指数MAE 58.79% | 无提升  1
训练:  0%|          | 0/51 [00:00<?, ?it/s]
验证:  0%|          | 0/13 [00:00<?, ?it/s]
Epoch  3/60 [ 30.9s lr=3.0e-04] loss 0.4320/0.4018 | Dice 坏死=0.6889 健康=0.7145
背景=0.9784 | 病情指数MAE  8.19% | 无提升  0 ← 新最佳
训练:  0%|          | 0/51 [00:00<?, ?it/s]
验证:  0%|          | 0/13 [00:00<?, ?it/s]
Epoch  4/60 [ 29.7s lr=3.0e-04] loss 0.3864/0.3641 | Dice 坏死=0.7322 健康=0.7145
背景=0.9846 | 病情指数MAE  6.29% | 无提升  0 ← 新最佳
训练:  0%|          | 0/51 [00:00<?, ?it/s]
验证:  0%|          | 0/13 [00:00<?, ?it/s]

```

Epoch 5/60 [30.0s lr=3.0e-04] loss 0.3328/0.3156 | Dice 坏死=0.7584 健康=0.9849 | 病情指数MAE 5.40% | 无提升 0 ← 新最佳
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]

Epoch 6/60 [29.7s lr=3.0e-04] loss 0.3045/0.3022 | Dice 坏死=0.6856 健康=0.9818 | 病情指数MAE 9.86% | 无提升 1
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]

Epoch 7/60 [29.2s lr=3.0e-04] loss 0.2827/0.3266 | Dice 坏死=0.7026 健康=0.9850 | 病情指数MAE 7.05% | 无提升 2
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]

Epoch 8/60 [29.5s lr=3.0e-04] loss 0.2699/0.2601 | Dice 坏死=0.7009 健康=0.9818 | 病情指数MAE 6.57% | 无提升 3
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]

Epoch 9/60 [29.8s lr=3.0e-04] loss 0.2507/0.2278 | Dice 坏死=0.7363 健康=0.9858 | 病情指数MAE 7.71% | 无提升 4
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]

Epoch 10/60 [30.1s lr=3.0e-04] loss 0.2329/0.2623 | Dice 坏死=0.7240 健康=0.9866 | 病情指数MAE 6.20% | 无提升 5
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]

Epoch 11/60 [29.5s lr=3.0e-04] loss 0.2221/0.2622 | Dice 坏死=0.6683 健康=0.9759 | 病情指数MAE 8.31% | 无提升 6
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]

Epoch 12/60 [30.2s lr=3.0e-04] loss 0.2164/0.2176 | Dice 坏死=0.7571 健康=0.9875 | 病情指数MAE 7.03% | 无提升 7
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]

Epoch 13/60 [30.8s lr=3.0e-04] loss 0.2020/0.1936 | Dice 坏死=0.7670 健康=0.9871 | 病情指数MAE 5.27% | 无提升 0 ← 新最佳
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]

Epoch 14/60 [29.9s lr=3.0e-04] loss 0.1979/0.1928 | Dice 坏死=0.7849 健康=0.9879 | 病情指数MAE 4.49% | 无提升 0 ← 新最佳
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]

Epoch 15/60 [29.5s lr=3.0e-04] loss 0.1993/0.1974 | Dice 坏死=0.7546 健康=0.9852 | 病情指数MAE 5.41% | 无提升 1
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]

Epoch 16/60 [30.1s lr=3.0e-04] loss 0.2072/0.2156 | Dice 坏死=0.7065 健康=0.9855 | 病情指数MAE 8.95% | 无提升 2
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]

Epoch 17/60 [29.7s lr=3.0e-04] loss 0.1910/0.1906 | Dice 坏死=0.7608 健康=0.9888 | 病情指数MAE 7.10% | 无提升 3
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]

Epoch 18/60 [30.0s lr=3.0e-04] loss 0.1869/0.1782 | Dice 坏死=0.7942 健康=0.9883 | 病情指数MAE 5.01% | 无提升 0 ← 新最佳
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]

Epoch 19/60 [30.4s lr=3.0e-04] loss 0.1796/0.1748 | Dice 坏死=0.7950 健康=0.9
背景=0.9895 | 病情指数MAE 4.91% | 无提升 0 ← 新最佳
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 20/60 [30.0s lr=3.0e-04] loss 0.1856/0.3886 | Dice 坏死=0.5180 健康=0.8
背景=0.9745 | 病情指数MAE 20.82% | 无提升 1
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 21/60 [29.8s lr=3.0e-04] loss 0.1904/0.1826 | Dice 坏死=0.7884 健康=0.9
背景=0.9888 | 病情指数MAE 4.92% | 无提升 2
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 22/60 [29.8s lr=3.0e-04] loss 0.1812/0.1784 | Dice 坏死=0.7871 健康=0.9
背景=0.9897 | 病情指数MAE 4.86% | 无提升 3
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 23/60 [31.0s lr=3.0e-04] loss 0.1702/0.1964 | Dice 坏死=0.7902 健康=0.9
背景=0.9842 | 病情指数MAE 4.61% | 无提升 4
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 24/60 [29.5s lr=3.0e-04] loss 0.1746/0.1931 | Dice 坏死=0.7686 健康=0.9
背景=0.9897 | 病情指数MAE 7.33% | 无提升 5
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 25/60 [30.3s lr=3.0e-04] loss 0.1780/0.1839 | Dice 坏死=0.7918 健康=0.9
背景=0.9858 | 病情指数MAE 5.33% | 无提升 6
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 26/60 [30.1s lr=1.5e-04] loss 0.1684/0.1637 | Dice 坏死=0.8051 健康=0.9
背景=0.9901 | 病情指数MAE 4.66% | 无提升 0 ← 新最佳
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 27/60 [29.1s lr=1.5e-04] loss 0.1655/0.1692 | Dice 坏死=0.8026 健康=0.9
背景=0.9900 | 病情指数MAE 4.46% | 无提升 1
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 28/60 [29.7s lr=1.5e-04] loss 0.1638/0.1542 | Dice 坏死=0.8181 健康=0.9
背景=0.9905 | 病情指数MAE 4.69% | 无提升 0 ← 新最佳
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 29/60 [30.2s lr=1.5e-04] loss 0.1602/0.1523 | Dice 坏死=0.8173 健康=0.9
背景=0.9904 | 病情指数MAE 4.42% | 无提升 1
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 30/60 [30.5s lr=1.5e-04] loss 0.1532/0.1576 | Dice 坏死=0.8009 健康=0.9
背景=0.9900 | 病情指数MAE 5.54% | 无提升 2
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 31/60 [30.4s lr=1.5e-04] loss 0.1550/0.1507 | Dice 坏死=0.8234 健康=0.9
背景=0.9896 | 病情指数MAE 4.15% | 无提升 0 ← 新最佳
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 32/60 [29.5s lr=1.5e-04] loss 0.1607/0.1501 | Dice 坏死=0.8233 健康=0.9
背景=0.9897 | 病情指数MAE 3.91% | 无提升 1
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]

Epoch 33/60 [30.6s lr=1.5e-04] loss 0.1503/0.1538 | Dice 坏死=0.8190 健康=0.9905 | 病情指数MAE 3.94% | 无提升 2
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]

Epoch 34/60 [29.2s lr=1.5e-04] loss 0.1527/0.1487 | Dice 坏死=0.8228 健康=0.9908 | 病情指数MAE 4.33% | 无提升 3
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]

Epoch 35/60 [30.8s lr=1.5e-04] loss 0.1471/0.1453 | Dice 坏死=0.8276 健康=0.9901 | 病情指数MAE 3.84% | 无提升 0 ← 新最佳
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]

Epoch 36/60 [30.2s lr=1.5e-04] loss 0.1481/0.1457 | Dice 坏死=0.8233 健康=0.9904 | 病情指数MAE 4.27% | 无提升 1
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]

Epoch 37/60 [29.7s lr=1.5e-04] loss 0.1504/0.1457 | Dice 坏死=0.8238 健康=0.9902 | 病情指数MAE 4.10% | 无提升 2
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]

Epoch 38/60 [30.1s lr=1.5e-04] loss 0.1492/0.1397 | Dice 坏死=0.8358 健康=0.9909 | 病情指数MAE 3.85% | 无提升 0 ← 新最佳
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]

Epoch 39/60 [29.9s lr=1.5e-04] loss 0.1486/0.1425 | Dice 坏死=0.8294 健康=0.9909 | 病情指数MAE 3.86% | 无提升 1
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]

Epoch 40/60 [30.5s lr=1.5e-04] loss 0.1481/0.1549 | Dice 坏死=0.8207 健康=0.9910 | 病情指数MAE 3.84% | 无提升 2
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]

Epoch 41/60 [29.7s lr=1.5e-04] loss 0.1485/0.1457 | Dice 坏死=0.8270 健康=0.9900 | 病情指数MAE 3.95% | 无提升 3
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]

Epoch 42/60 [30.0s lr=1.5e-04] loss 0.1473/0.1473 | Dice 坏死=0.8232 健康=0.9900 | 病情指数MAE 4.03% | 无提升 4
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]

Epoch 43/60 [30.6s lr=1.5e-04] loss 0.1406/0.1558 | Dice 坏死=0.8145 健康=0.9904 | 病情指数MAE 3.83% | 无提升 5
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]

Epoch 44/60 [29.6s lr=1.5e-04] loss 0.1475/0.1478 | Dice 坏死=0.8226 健康=0.9896 | 病情指数MAE 3.85% | 无提升 6
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]

Epoch 45/60 [29.4s lr=7.5e-05] loss 0.1397/0.1370 | Dice 坏死=0.8326 健康=0.9904 | 病情指数MAE 3.63% | 无提升 7
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]

Epoch 46/60 [30.9s lr=7.5e-05] loss 0.1411/0.1384 | Dice 坏死=0.8363 健康=0.9907 | 病情指数MAE 3.74% | 无提升 0 ← 新最佳
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]

Epoch 47/60 [29.6s lr=7.5e-05] loss 0.1379/0.1402 | Dice 坏死=0.8382 健康=0.9
背景=0.9909 | 病情指数MAE 3.48% | 无提升 0 ← 新最佳
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 48/60 [29.3s lr=7.5e-05] loss 0.1363/0.1403 | Dice 坏死=0.8256 健康=0.9
背景=0.9901 | 病情指数MAE 4.32% | 无提升 1
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 49/60 [29.9s lr=7.5e-05] loss 0.1358/0.1390 | Dice 坏死=0.8315 健康=0.9
背景=0.9897 | 病情指数MAE 3.43% | 无提升 2
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 50/60 [30.6s lr=7.5e-05] loss 0.1399/0.1433 | Dice 坏死=0.8317 健康=0.9
背景=0.9905 | 病情指数MAE 3.45% | 无提升 3
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 51/60 [29.6s lr=7.5e-05] loss 0.1347/0.1389 | Dice 坏死=0.8343 健康=0.9
背景=0.9907 | 病情指数MAE 3.55% | 无提升 4
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 52/60 [30.3s lr=3.7e-05] loss 0.1303/0.1301 | Dice 坏死=0.8434 健康=0.9
背景=0.9913 | 病情指数MAE 3.50% | 无提升 0 ← 新最佳
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 53/60 [30.4s lr=3.7e-05] loss 0.1311/0.1348 | Dice 坏死=0.8345 健康=0.9
背景=0.9906 | 病情指数MAE 3.73% | 无提升 1
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 54/60 [29.4s lr=3.7e-05] loss 0.1318/0.1341 | Dice 坏死=0.8412 健康=0.9
背景=0.9908 | 病情指数MAE 3.50% | 无提升 2
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 55/60 [29.4s lr=3.7e-05] loss 0.1318/0.1352 | Dice 坏死=0.8389 健康=0.9
背景=0.9908 | 病情指数MAE 3.40% | 无提升 3
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 56/60 [30.7s lr=3.7e-05] loss 0.1289/0.1348 | Dice 坏死=0.8307 健康=0.9
背景=0.9909 | 病情指数MAE 4.22% | 无提升 4
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 57/60 [29.6s lr=3.7e-05] loss 0.1322/0.1339 | Dice 坏死=0.8358 健康=0.9
背景=0.9911 | 病情指数MAE 4.20% | 无提升 5
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 58/60 [29.5s lr=3.7e-05] loss 0.1284/0.1341 | Dice 坏死=0.8369 健康=0.9
背景=0.9905 | 病情指数MAE 3.61% | 无提升 6
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 59/60 [29.9s lr=1.9e-05] loss 0.1277/0.1330 | Dice 坏死=0.8396 健康=0.9
背景=0.9910 | 病情指数MAE 3.36% | 无提升 7
训练: 0%| | 0/51 [00:00<?, ?it/s]
验证: 0%| | 0/13 [00:00<?, ?it/s]

Epoch 60/60 [30.1s lr=1.9e-05] loss 0.1292/0.1331 | Dice 坏死=0.8412 健康=0.9912 | 病情指数MAE 3.44% | 无提升 8

完成!最佳坏死Dice: 0.8434 | 耗时 30.0 分钟

最佳模型: /root/cassava/checkpoints/best_model.pth

```
In [13]: # 验证集分级混淆矩阵
ckpt = torch.load(best_path, map_location=DEVICE, weights_only=False)
model.load_state_dict(ckpt["model_state_dict"])
model.eval()
print(f"已载入最佳模型 (epoch {ckpt['epoch']}, 坏死Dice {ckpt['necrosis_dice']})")

conf = np.zeros((len(SEVERITY_ORDER), len(SEVERITY_ORDER)), dtype=int)
pairs = []

with torch.no_grad():
    for images, masks in tqdm(val_loader, desc="评估分级", leave=False):
        images = images.to(DEVICE)
        with torch.autocast(device_type=DEVICE.type, enabled=USE_AMP):
            preds = torch.argmax(model(images), dim=1)

        for p, t in zip(preds.cpu().numpy(), masks.numpy()):
            sp, st = severity_from_labels(p), severity_from_labels(t)
            if sp is None or st is None:
                continue
            pairs.append((st, sp))
            conf[SEVERITY_ORDER.index(grade_severity(st)),
                  SEVERITY_ORDER.index(grade_severity(sp))] += 1

header = "真值↓ / 预测→"
print(f"\n{header:<16}" + "".join(f"{SEVERITY_LABELS[k]:<18}" for k in SEVERITY_ORDER))
for i, k in enumerate(SEVERITY_ORDER):
    row_sum = conf[i].sum()
    recall = conf[i, i] / row_sum * 100 if row_sum else 0
    print(f"{SEVERITY_LABELS[k]:<16}" + "".join(f"{n:<18}" for n in conf[i]))

acc = np.trace(conf) / conf.sum() * 100
print(f"\n分级准确率: {acc:.1f}% ({np.trace(conf)}/{conf.sum()})")
```

已载入最佳模型 (epoch 52, 坏死Dice 0.8434)

评估分级: 0% | 0/13 [00:00<?, ?it/s]

真值↓ / 预测→	轻度 (<20%)	重度 (≥20%)	召回率
轻度 (<20%)	131	7	94.9%
重度 (≥20%)	4	66	94.3%

分级准确率: 94.7% (197/208)